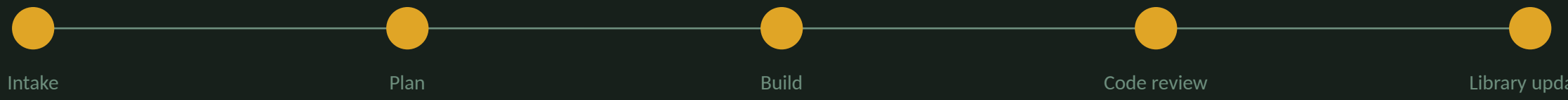
PROPOSAL · NOVEMBER 2026

Bagheera

Every feature makes the next one smarter

A delivery harness on Claude Code · for product and engineering leaders



Today, every feature starts from zero

FEATURE 1

Decisions are made in a chat thread or a meeting

FEATURE 2

The same questions get asked and answered again

FEATURE 3

The same mistakes get made again

Lost in chat threads, meetings, old docs and people's heads

Claude knows none of it
Every session starts without our standards, services or past decisions.

PMs are outside the loop
They see a ticket, then a demo weeks later; questions wait for meetings.

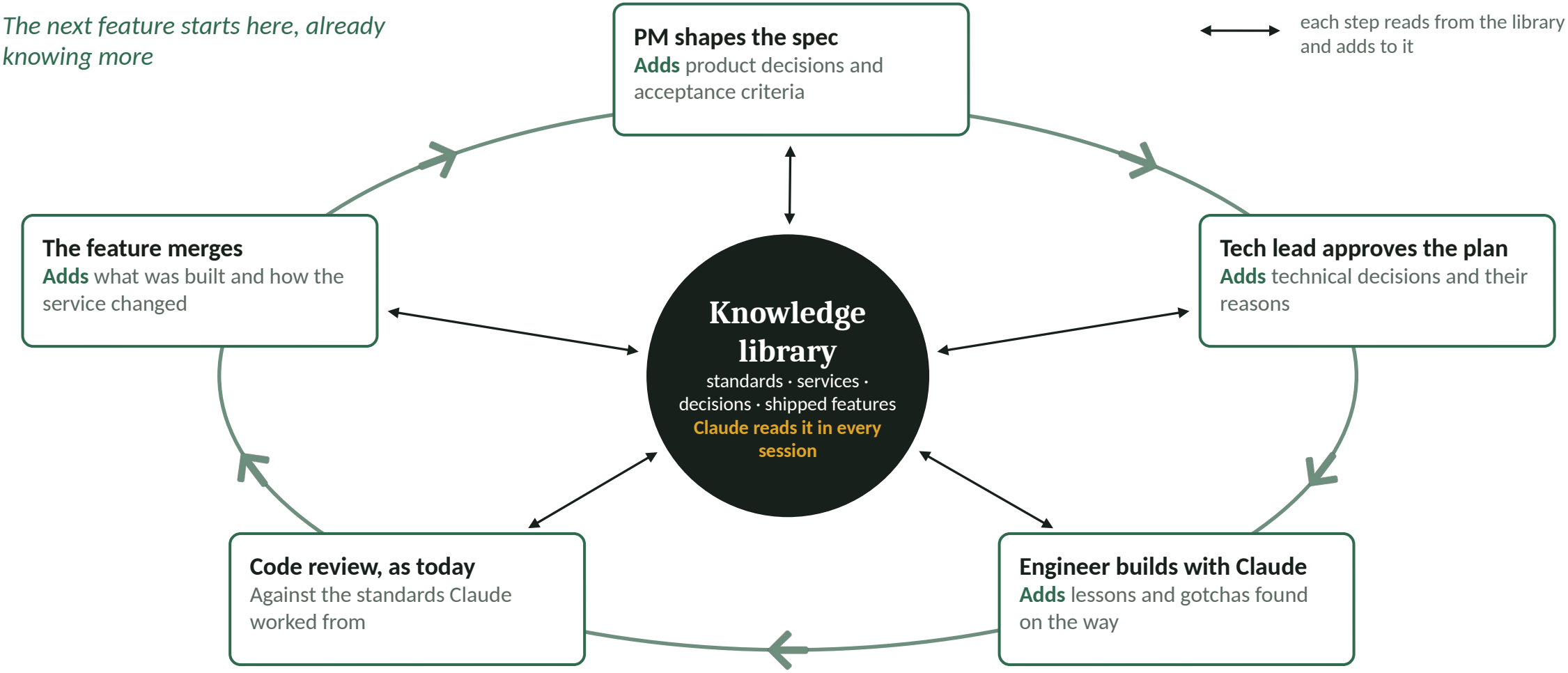
Everyone works their own way
Each team and engineer reads the standards differently.

Today versus with Bagheera

	Today	With Bagheera
Knowledge	Standards, service details and past decisions live in heads, old docs and chat logs	One knowledge library for the org that every approved change adds to; Claude reads it in every session
Standards	Each team reads them differently; reviewers catch issues late and unevenly	Written once by the architecture council, in Claude's context as it works; dangerous actions blocked
Developer path	Everyone works with AI differently; specs and plans stay in private chats	The same few commands in every repo, ticket to merged code, next step always clear
PM feedback	A ticket, then a demo weeks later; questions wait for meetings	PMs answer questions and approve the spec and its acceptance criteria in the PR, within hours
Leadership view	Status, bottlenecks and AI cost are assembled by hand	Every approval and override on record in the PR; a dashboard comes with org adoption

Every feature makes the next one smarter

The next feature starts here, already knowing more



Decided once, reused by every feature

Who	In their normal work…	…the library keeps
PM	Answers the open questions and approves the spec and its acceptance criteria	Product decisions and acceptance criteria
Tech lead	Approves the plan, with its design and phases	Technical decisions, with the reasons
Engineer	Proposes design choices, and hits the odd gotcha while building	Proposed decisions and lessons learned
Architecture council	Writes each standard once and approves every change to it	Standards that apply in every repo
Every merge	The developer runs the library update; an agent drafts the page changes	Service pages and feature history, kept current

No extra step: it's captured from approvals people already give. A person approves every library change, and each approval names who gave it.

Claude starts every session knowing us

New session · payments-api example

Loaded automatically before the first prompt

SERVICE PAGE

Charges cards and retries failures. Depends on ledger-svc. Gotcha: the card provider times out after 30 seconds.

THE INDEX

One line for every page: standards for Go, API design, secrets and payments security, opened as the work needs them

PAST DECISIONS

Every write carries an idempotency key, decided in PAY-301 with the reasons

YOUR FEATURE

PAY-412 · plan approved · next: /bagheera:build, phase 1 of 3

Code that fits the service

It knows what the service does, what it depends on and where it breaks.

Standards followed from the first draft

The org's standards for this language and service are a page away while it designs and writes.

Settled questions stay settled

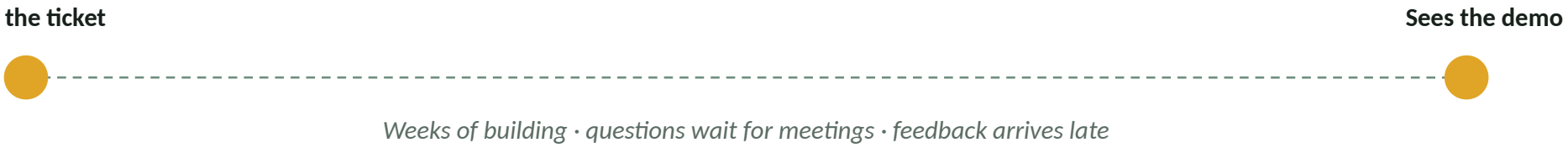
Past decisions come with their reasons, so they aren't argued again or quietly undone.

Old pages are hints, not facts

A page past its shelf life is checked against the code; when pages disagree, the newer wins.

From SDLC to PDLC

Today: SDLC
PM at the edges



With Bagheera
PM inside each key step

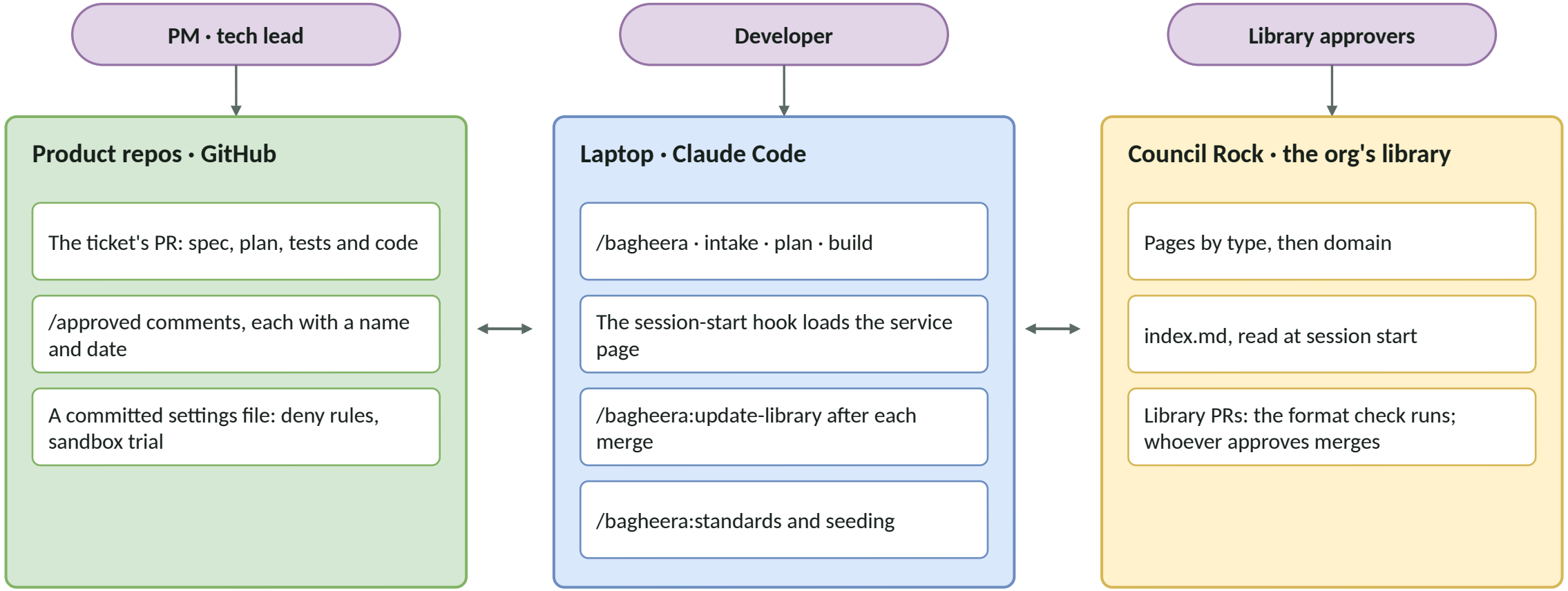


Feedback in hours, not weeks
PMs review where the work happens, as it happens.

Fewer clarification meetings
Questions are answered inline, in the pull request.

Product decisions kept
Every answer and approval adds to the knowledge library.

The MVP: what one team runs



From outside the team: the org's plugin marketplace installs the plugin · Jira and Confluence supply tickets and standards pages · the platform team builds the plugin and the settings template

Full diagram: docs/architecture-mvp.drawio

Built from tools we already use

2

repos: the plugin and the org's knowledge library

6

commands, adapted from Spec Kit's templates

1

workflow: the library's format check

0

new servers, and nothing from org admins

What we configure, not build

Claude Code	One plugin for every engineer, from the org's plugin marketplace
GitHub Spec Kit	The stage commands' method and templates, adapted once
GitHub	Pull requests, approval comments, branch rules and the format check
Jira and Confluence	Tickets in; standards seeded from the pages we already have

Our first attempt had too many moving parts. This one is sized for a platform team of two.

Pilot first, then grow team by team

1

Build the MVP
Six commands, the library and its format check, tested end to end

2

Pilot
One team of 6–8 engineers, at least 10 real tickets

3

More teams
Each adopts it alone, 2–3 at a time; team additions as their signals show

4

Org adoption
CI, managed settings and the dashboard, each when its signal shows

About 6 weeks to the pilot · 2 platform engineers · 1 pilot team of 6–8 engineers

How we'll know it works

≥ 10

tickets taken through intake, plan and build, each with its spec and plan approved before Build

100%

of those tickets update the library, approved by their tech lead or PM

After the pilot
Efficiency: response times, meetings, weekly use and cost per merged PR. Their targets are set then.

Where it goes: the target state

STAYS AS IT IS

The MVP

- The commands people type
- Approvals as PR comments, on the record
- One library for the org, and its rules
- Standards owned by the architecture council

Nothing for people to relearn as the org adopts it.

Added by a team, each when its signal shows

Verify: an automated review of each PR · the standards hook · separate architecture, test-plan and demo stages · criteria results from CI · the Jira comment · checks for planted text and sources on library pages

Added at org adoption, each when its signal shows

CI and a GitHub App
The review on every PR; the library update on a schedule

Claude sign-in for CI
Short-lived access, CI's own workspace, spend caps

Managed settings
Deny rules locked and the plugin on, for everyone

Leadership dashboard
Every story's stage and who it waits on

Weekly summary · Slack
What shipped, what's stuck; review reminders

Page ownership
domains.yaml: each domain's repos and owners

Full diagram: docs/architecture-target.drawio · each part's signal: docs/proposals/org-adoption.md

What we need from you

1. **A pilot team and at least 10 real tickets**
   on Macs, with a PM and tech lead who approve in pull requests

2. **GitHub access for PMs**
   to comment on and approve their team's pull requests

3. **Our Claude plan and model access confirmed**
   Claude Code seats for the pilot engineers

4. **The architecture council's time**
   to seed the first standards from Confluence and approve them

5. **Backing for the pilot rule**
   we expand only once the team uses the approvals rather than skipping them

Next: the owner reviews the plan, then the platform team builds the MVP.